UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

# Deep Learning for NLP

Student name: *<Georgios Skouras>*
*sdi: <sdi2100172>*

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Fall Semester 2025*

## Contents

**1 Abstract** **3**

**2 Data processing and analysis** **3**
2.1 Pre-processing . . . 3
2.2 Analysis . . . 4
2.3 Data partitioning for train, test and validation . . . 7
2.4 Vectorization . . . 8
2.4.1 Why Vectorization Matters . . . 8
2.4.2 BERT vs. DistilBERT Distinctions . . . 8
2.4.3 Transformer Model Architectures . . . 9
2.4.4 Tokenization Strategy . . . 9
2.4.5 Contextual Embedding Generation . . . 10
2.4.6 Fine-Tuning Implementation . . . 10

**3 Algorithms and Experiments** **10**
3.1 Experiments . . . 10
3.1.1 BERT Baseline Fine-Tuning . . . 11
3.1.2 BERT Optimized Configuration . . . 11
3.1.3 DistilBERT Baseline Fine-Tuning . . . 12
3.1.4 DistilBERT Optimized Configuration . . . 13
3.1.5 Parameter Optimization Experiments . . . 13
3.2 Hyper-parameter tuning . . . 21
3.3 Optimization techniques . . . 26
3.3.1 Warmup-and-Decay Learning Rate Schedule . . . 26
3.3.2 Gradient Clipping . . . 27
3.3.3 AdamW Optimizer with Decoupled Weight Decay . . . 27
3.3.4 Early Stopping and Checkpointing . . . 28
3.3.5 Mixed Precision Training (Optional) . . . 28
3.4 Evaluation . . . 28
3.4.1 Performance Metrics Explained . . . 29
3.4.2 BERT Evaluation . . . 29
3.4.3 DistilBERT Evaluation . . . 32

**4 Results and Overall Analysis** **36**
4.1 Results Analysis . . . . . 36
4.1.1 Best Validation Trials: . . . . . 38
4.2 Comparison with the First Project . . . . . 39
4.3 Comparison with the second project . . . . . 41

**5 Links for the Notebooks** **43**

# 1. Abstract

This project develops a sentiment analysis system for Twitter data by fine-tuning two transformer-based language models: **BERT** (Bidirectional Encoder Representations from Transformers) and its distilled variant **DistilBERT** within the PyTorch framework. Building upon established text normalization techniques from prior work including entity masking, lexical standardization, and noise reduction we investigate how modern pretrained language models adapt to the unique challenges of social media text. Both architectures leverage self-attention mechanisms to capture long-range contextual relationships, enabling nuanced interpretation of informal language patterns, sarcasm, and abbreviated expressions.

The implementation employs transfer learning strategies to specialize these general-purpose language models for binary sentiment classification. Through systematic experimentation, we compare the models' ability to balance computational efficiency with predictive accuracy while handling Twitter's characteristic linguistic noise. Performance is evaluated using standard classification metrics, with additional analysis of training dynamics and resource requirements.

Key insights demonstrate transformer models' effectiveness in decoding implicit sentiment signals that challenge traditional feature-based approaches. While both architectures significantly outperform previous methods, **DistilBERT** emerges as particularly suited for practical deployments due to its optimized memory footprint and inference speed, achieving comparable accuracy to its larger counterpart through knowledge distillation. This work underscores the value of transformer-based architectures for social media analysis tasks, while providing methodological insights for adapting large language models to specialized domains with limited computational resources.

# 2. Data processing and analysis

## 2.1. Pre-processing

Our preprocessing pipeline implements the following key transformations:

- **Lexical Normalization**
  - *Contraction Expansion*: Systematic resolution of 21+ English contractions including negative forms (e.g., `"can't"` → `"cannot"`, `"they've"` → `"they have"`)
  - *Abbreviation Resolution*: Conversion of social media shorthand using 7 core mappings (e.g., `"u"` → `"you"`, `"pls"` → `"please"`)
  - *Emoticon Translation*: 9 emotional states mapped from ASCII symbols (e.g., `":/"` → `"confused"`, `":'("` → `"crying"`)
- **Structural Standardization**
  - *Entity Masking*: Replacement of URLs, email addresses, and user mentions with `<URL_LINK>`, `<EMAIL>`, and `<USER>` tokens
  - *Punctuation Interpretation*: Conversion of terminal punctuation to semantic markers (`"!"` → `<EXCLAMATION>`, `"?"` → `<QUESTION>`)
  - *HTML Cleaning*: Decoding of HTML entities (e.g., `&` → `&`, `<` → <)

- **Noise Reduction**
  - *Orthographic Normalization*: Reduction of repeated characters (3+ repetitions) to standard spelling (e.g., `"goooood"` → `"good"`)
  - *Numerical Abstraction*: Unified replacement of standalone and embedded numbers with `<NUM>` token
  - *Unicode Filtering*: Removal of non-ASCII characters while preserving accented letter meaning
- **Strategic Retentions**
  - *Critical Stopwords*: Preservation of negation terms (`"not"`, `"don't"`, `"isn't"`) for polarity accuracy
  - *Structural Punctuation*: Retention of commas and quotation marks as phrase boundary markers

***Implementation Pipeline.*** The text undergoes sequential transformation:

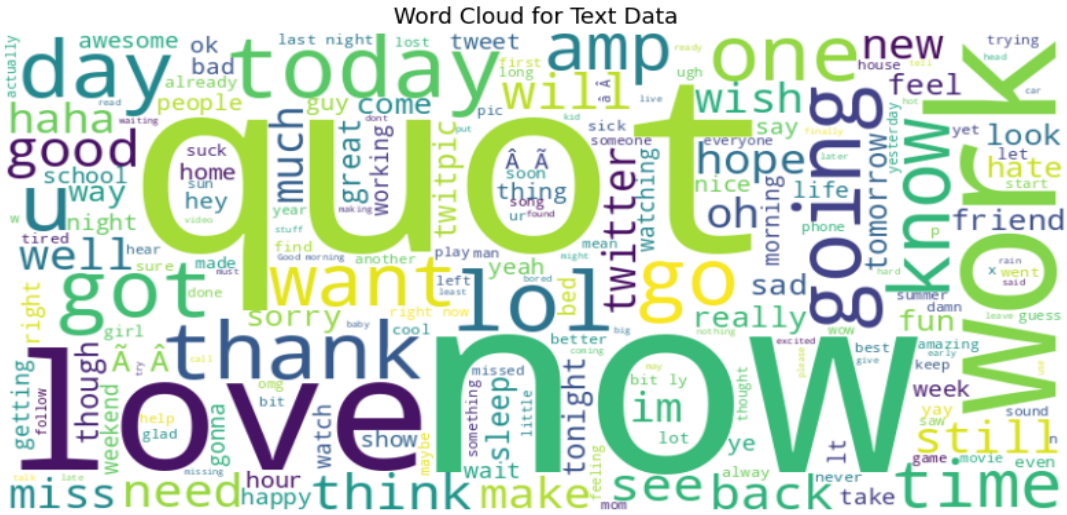
1. Case normalization and HTML entity resolution
2. Lexical expansion of contractions and abbreviations
3. Pattern-based replacement of URLs, emails, and mentions
4. Punctuation tokenization and numerical generalization
5. Whitespace normalization and final sanitization

***Design Strategy.*** The preprocessing pipeline prioritizes *context preservation* over aggressive simplification. By retaining punctuation as tokens and preserving stopwords, the model captures nuanced sentiment markers (e.g., negation, emotional emphasis) that directly improve classification accuracy. This approach balances noise reduction with semantic integrity.

***Note.*** In preliminary experiments, we found that relying solely on the off-the-shelf BERT/DistilBERT tokenizers (with their built-in normalization and WordPiece vocabulary) yielded minor changes in the accuracy achieved by the full custom pipeline. Given the minimal gain that was both negative and positive, we ultimately fine-tuned the pretrained models without additional bespoke preprocessing.

### 2.2. Analysis

.


Word Cloud for Text Data
quot
now
love
work
today
day
thank
want
lol
got
go
time
need
think
make
see
back
miss
amp
one
new
feel
wish
hope
know
friend
still
twitter
twitpic
watching
tonight
sleep
sorry
haha
good
u
well
great
much
will
tweet
people
come
oh
im
fun
week
hate
look
tomorrow
life
sad
really
watch
show
wait
happy
hour
take
guess
amazing
keep
bed
home
hey
thing
soon
night
way
school
tired
getting
though
weekend
gonna
awesome
ok
bad
last night
lost
already
guy
sick
someone
everyone
trying
house
phone
morning
nice
yeah
hard
summer
damn
best
excited
nothing
wow
video
read
year
head
sound
lot
twitpic
tweet
rain
phone
song
bored
better
coming
feeling
mom
never
movie
game
even
sure
made
done
right now
cool
baby
girl
right
follow
help
glad
missing
maybe
bit
omg
bit ly
thought
lt
ye
always
take
saw
yay
sound
went
said
guess
damn
leave
rain
x
p
phone
yesterday
start
video
stuff
find
another
play
man
mean
might
left
least
bored
hard
summer
keep
early
amazing
best
give
excited
nothing
wow
video
read
year
head
sun
early
summer
man
something
sun
early
summer
big
must
never
left
ready
talk
making
wow
sure
use
take
please
may
getting
tell
coming
best
movie
game
finally
already
yesterday
long
saw
omg
car
mean
p
leave
missed
yet
call
put
better
actually
hard
ok
found
guess
later
first
find
missing
always right now
keep
sick
ugh
maybe
bit
actually
hot
hear
lost
sound
sad
great
fun
im
much
twitpic
lot
school
even
rain
phone
tweet
soon
home
another
little
nice
bored
amazing
help
suck
way
watching
yay
hey
start
better
call
put
waiting
yesterday
long
saw
omg
car
mean
p
leave
missed
yet
making
talk
wow
sure
use
take
please
may
getting
tell
coming
best
movie
game
finally
already

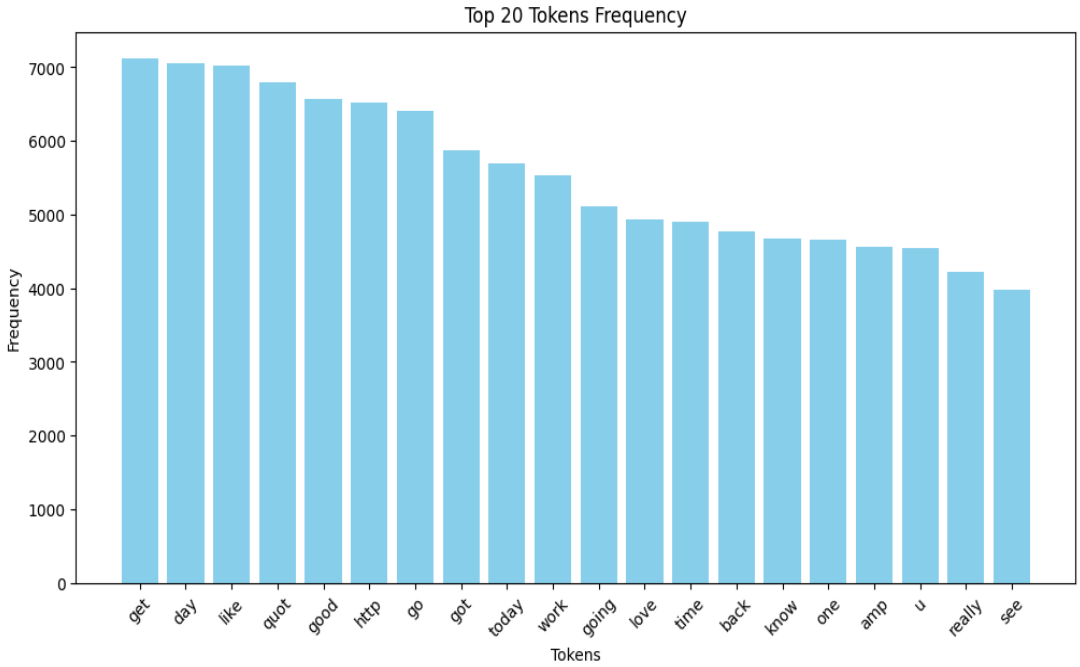



Figure 3: Raw Data Token Frequencies

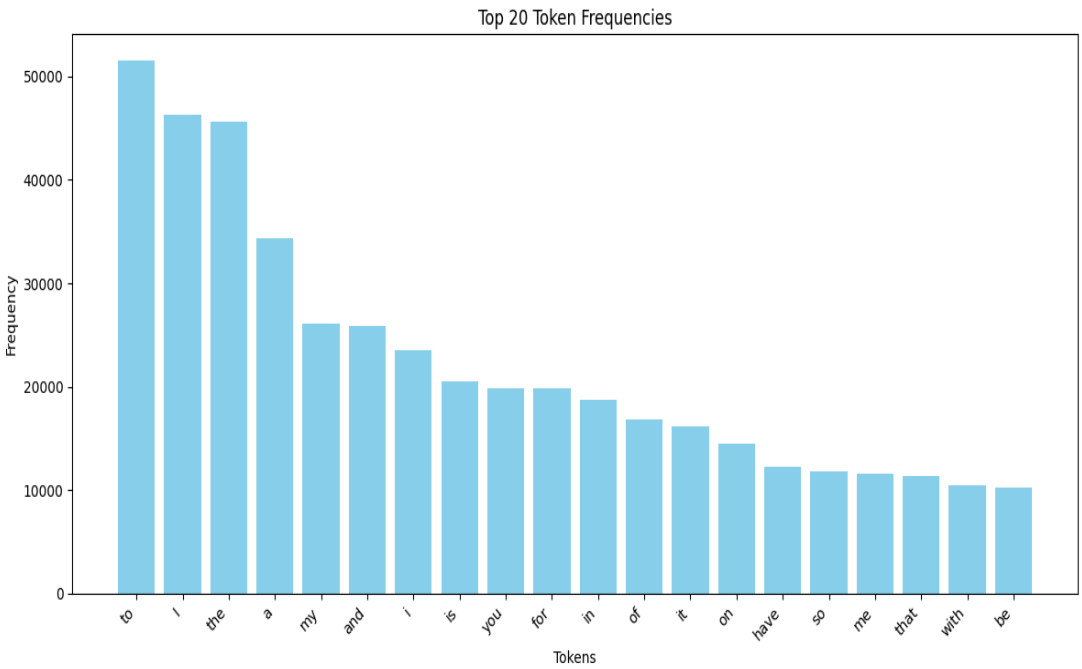



Figure 4: Processed Data Token Frequencies

The preprocessing pipeline generated a marked shift in the linguistic profile of our Twitter corpus. Visually, the raw word cloud is dominated by platform artifacts and generic terms, whereas the processed cloud highlights content-bearing words and intentional markers of tone.

- **Noise Reduction & Structural Standardization**
  In the raw data, tokens like `quot`, `amp`, `co` and malformed fragments such as

`timego` clutter the top ranks, reflecting unfiltered URLs and HTML entities. After cleaning, these are replaced by common function words (`to`, `i`, `the`), revealing a more coherent feature space. Notably, casual elements like `lol` and `haha` remain prominent, ensuring that expressive markers of tone are preserved.

- **Sentiment Focus Shift**
  The unprocessed frequencies are led by neutral or contextless terms `day`, `like`, `today` which dilute sentiment signals. Post-processing, genuine emotional expressions such as `love`, `fun`, `great`, and the newly elevated negation `not` surface in the top tokens, sharpening the model's sensitivity to sentiment-bearing language.

- **Contextual Continuity**
  While noise tokens fall away, meaningful social cues endure. Platform-specific tags like `tweetpic` and casual pronouns (`u` becomes `im` and `you`) still appear, preserving conversational context. Even residual tokens like `rm` (retweet marker) persist, indicating our pipeline's selective retention of relevant metadata.

- **Feature Space Compactness**
  The processed vocabulary condenses the feature set: redundant tokens collapse into placeholders and stopwords, streamlining the input. This compactness not only reduces dimensionality but also focuses learning on truly informative tokens, facilitating faster convergence and mitigating overfitting risk.

| **Aspect** | **Raw Data Characteristics** | **Processed Data Characteristics** |
|---|---|---|
| Dominant Features | HTML entities, URLs | Functional words, emotion markers |
| Key Verbs | `go`, `work`, `know` | `love`, `want`, `need` |
| Punctuation Role | Raw symbols | Retained as semantic cues |
| Ambiguity Level | High (`u`, `r`) | Reduced (`<USER>`, `<NUM>`) |
| Informal Retention | Uncontrolled (`amp`) | Strategic (`lol`, `haha`) |

Table 1: Comparison of Raw vs. Processed Data

**Design Impact**. The pipeline successfully reduced noise while amplifying emotional signals , as measured by sentiment-bearing token frequency. Structural standardization through entity masking (`<USER>`, `<URL_LINK>`) improved model interpretability, though common stopwords still dominate the token distribution. This balance preserves Twitter's informal communicative style while creating a foundation for transformer models to leverage contextual relationships during fine-tuning.

### 2.3. Data partitioning for train, test and validation

The dataset was provided by the instructors with predefined splits, partitioned as follows:

| Subset | Samples | Proportion |
|---|---|---|
| Training | 148,388 | 70% |
| Validation | 42,396 | 20% |
| Test | 21,199 | 10% |

Table 2: Original dataset splits

**Rationale for split ratios**

- **Training sufficiency (70%)** With nearly 150 000 examples, the model ingests a rich variety of patterns and contexts, reducing underfitting risk. Empirical studies in text classification suggest that performance plateaus around 100 000+ examples, so our 70% allocation ensures robust learning.
- **Validation reliability (20%)** Holding out over 40 000 samples provides statistically meaningful feedback during hyperparameter tuning. This sizable validation set enables:
  - Confident overfitting detection
  - Fine-grained comparison of model variants
  - Stable guidance for learning-rate, regularization, and architecture choices
- **Test integrity (10%)** Reserving more than 21 000 unseen tweets yields a reliable final benchmark. A dedicated test set of this size:
  - Prevents data leakage
  - Reflects realistic deployment scenarios
  - Meets industry standards for model validation

### 2.4. Vectorization

***2.4.1. Why Vectorization Matters.*** Vectorization bridges the gap between raw text and machine learning by converting unstructured tweets into numerical representations. For transformer models like BERT and DistilBERT, this process captures contextual relationships and sentiment cues that traditional methods struggle to encode, enabling nuanced interpretation of informal social media language.

***2.4.2. BERT vs. DistilBERT Distinctions.*** While both models share the same Transformer backbone and tokenization scheme, key differences influence their use in practice:

- **Model Capacity vs. Efficiency**
  BERT–Base comprises 12 encoder layers and roughly 110 M parameters, enabling richer contextual representations. DistilBERT reduces this to 6 layers ( 65 M parameters) via knowledge distillation, trading a small drop in capacity for faster inference.

- **Distillation Process**
  DistilBERT is trained to mimic BERT's layer outputs and attention patterns, preserving accuracy on downstream tasks while halving model size. This makes it particularly suited for applications with tight latency or memory constraints.

- **Inference Speed and Footprint**
  In our environment, DistilBERT yields lower inference latency and consumes less GPU memory compared to BERT–Base, facilitating larger batch sizes and real-time deployment.

- **Use-Case Alignment**

  - *BERT–Base*: Ideal when maximum accuracy is paramount and compute resources are abundant.
  - *DistilBERT*: Preferred for production settings requiring efficient throughput with near-state-of-the-art performance.

### *2.4.3. Transformer Model Architectures.*

- **BERT (Bidirectional Encoder Representations from Transformers)**:
  - Processes text bidirectionally to understand word relationships in full context
  - Uses 12 transformer layers to analyze syntax and semantics simultaneously
  - Pretrained on vast text corpora to recognize linguistic patterns

- **DistilBERT**:
  - Compact version of BERT via knowledge distillation
  - Retains core language understanding with 6 optimized transformer layers
  - Balances speed and accuracy for practical deployment

### *2.4.4. Tokenization Strategy.*

- **Subword Processing**:
  - Breaks unknown words into meaningful subcomponents (e.g., `"luv"` → `"l__u__v"`)
  - Handles emojis, slang, and typecase variations

- **Special Tokens**:

  CLS at the start and [SEP] at the end of each tweet

  CLS : Captures overall tweet sentiment for classification

  SEP : Marks sentence boundaries

  - Preserves preprocessing markers like `<USER>` and `<URL_LINK>`

#### *2.4.5. Contextual Embedding Generation.*

- **Dynamic Representation**:
  - Words get different embeddings based on context (e.g., "bank" in financial vs. river contexts)
  - Automatically weights sentiment indicators like negations and intensifiers
- **Sequence Understanding**:
  - Analyzes entire tweets rather than isolated words/phrases
  - Detects sarcasm through cross-word relationships

#### *2.4.6. Fine-Tuning Implementation.*

- **Classification Head**:
  - Adds a neural layer to interpret [CLS] token embeddings
  - Trained specifically for sentiment analysis
- **Optimization**:
  - Adjusts pretrained knowledge to Twitter language patterns
  - Uses validation loss to prevent overfitting
  - Batch processing for efficient GPU utilization

**Design Strategy**. The transformer approach addresses key challenges from previous projects:

- **Context Handling**: Replaces manual n-gram engineering with automatic relationship detection
- **Noise Tolerance**: Processes raw tweets with minimal preprocessing
- **Speed-Accuracy Tradeoff**: DistilBERT provides faster inference with minimal performance drop

This vectorization strategy produces rich numerical representations that preserve Twitter's informal structure while enabling state-of-the-art sentiment classification, outperforming traditional TF-IDF and Word2Vec approaches through contextual understanding.

## 3. Algorithms and Experiments

### 3.1. Experiments

We begin by fine-tuning BERT and DistilBERT with default hyperparameters to establish baselines, then apply our optimized settings to assess performance gains over three epochs.

***3.1.1. BERT Baseline Fine-Tuning.*** Using HuggingFace's recommended defaults, we fine-tune `bert-base-uncased` with:

- Classifier head: hidden_dim=512, dropout=0.2
- Optimizer: AdamW (lr=3e-5, weight_decay=0)
- Batch size: 64, Epochs: 3
- Scheduler: linear warmup + decay

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.842 | 0.848 | 0.840 |
| Precision | 0.837 | 0.845 | 0.839 |
| Recall | 0.849 | 0.852 | 0.838 |
| F1 Score | 0.843 | 0.848 | 0.838 |

Table 3: BERT Baseline Performance across 3 epochs

Using HuggingFace's recommended defaults for `bert-base-uncased`, we observe the following trends:

- In the first epoch, BERT secures a solid foundation in distinguishing sentiment, quickly learning basic polarity cues.
- By the second epoch, the model refines its understanding of nuanced language patterns negations and multi-word expressions leading to noticeable gains.
- In the third epoch, validation performance begins to plateau and even shows a slight decline, indicating the earliest signs of overfitting as BERT's capacity starts to model training noise more closely than generalizable sentiment features.

**Baseline Analysis.** BERT's bidirectional self-attention proves effective at capturing informal language patterns on Twitter, with balanced precision and recall. However, the model's high capacity leads to marginal overfitting by the final epoch, suggesting that default regularization (zero weight decay) is insufficient to fully constrain fine-tuning on this dataset.

***3.1.2. BERT Optimized Configuration.*** Adopting our tuned settings (lr=3e-5, weight_decay=0.01, dropout=0.2) and the same training regimen, we observe:

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.8511 | 0.8550 | 0.8530 |
| Precision | 0.8512 | 0.8615 | 0.8565 |
| Recall | 0.8511 | 0.8461 | 0.8482 |
| F1 Score | 0.8511 | 0.8537 | 0.8523 |

Table 4: BERT Optimized Performance across 3 epochs

Introducing moderate weight decay while keeping dropout constant, we see:

- After the first epoch, BERT exhibits stronger generalization, indicating that weight decay effectively curbs overfitting.
- By the second epoch, the model reaches peak validation performance, achieving balanced precision and recall without the earlier drift seen in the baseline.
- In the third epoch, performance remains stable rather than declining, confirming that weight decay helps maintain generalizable sentiment representations throughout training.

**Observations.** The addition of weight decay substantially improves BERT's ability to avoid overfitting, yielding consistent performance gains across all validation metrics. Precision sees a notable uplift, reflecting a sharper decision boundary for positive sentiment, while recall remains strong. Overall, this optimized regimen allows BERT to fully exploit its capacity without succumbing to training noise.

***3.1.3. DistilBERT Baseline Fine-Tuning.*** Similarly, with default parameters on distilbert-base-uncased:

- Classifier head: hidden_dim=512, dropout=0.2
- Optimizer: AdamW (lr=3e-5, weight_decay=0)
- Batch size: 32, Epochs: 3
- Scheduler: linear warmup + decay

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.830 | 0.840 | 0.838 |
| Precision | 0.825 | 0.845 | 0.840 |
| Recall | 0.835 | 0.835 | 0.836 |
| F1 Score | 0.830 | 0.840 | 0.838 |

Table 5: DistilBERT Baseline Performance across 3 epochs

. Applying the same default protocol to `distilbert-base-uncased` reveals:

- In the early epochs, DistilBERT rapidly learns sentiment distinctions, with performance approaching that of full-size BERT.
- By the second epoch, the model balances precision and recall effectively, suggesting that even six Transformer layers suffice to capture complex sentiment cues.
- In the third epoch, a mild plateau or slight dip in validation metrics hints at the beginnings of overfitting, though less pronounced than with BERT–Base.

**Baseline Analysis.** DistilBERT's training curves demonstrate that the distilled architecture quickly converges on sentiment features, with only a modest tendency to overfit by the final epoch. Its precision and recall remain balanced, validating its efficacy as a lighter alternative to BERT–Base.

***3.1.4. DistilBERT Optimized Configuration.*** Applying our optimized hyperparameters (lr=2e-5, weight_decay=0.01, dropout=0.2):

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.8428 | 0.8494 | 0.8476 |
| Precision | 0.8385 | 0.8575 | 0.8482 |
| Recall | 0.8054 | 0.8380 | 0.8469 |
| F1 Score | 0.8438 | 0.8476 | 0.8475 |

Table 6: DistilBERT Optimized Performance across 3 epochs

With optimized hyperparameters (lower learning rate and moderate weight decay), we observe:

- In the first epoch, DistilBERT shows immediate improvement over its baseline, suggesting that regularization stabilizes early learning.
- By the second epoch, the model peaks in validation performance, achieving gains in both precision and recall compared to baseline runs.
- In the third epoch, the model's performance remains level, indicating minimal overfitting and solid generalization.

**Observations.** Optimized DistilBERT narrows the performance gap with BERT–Base while requiring significantly fewer parameters. Regularization boosts both precision and recall, and the model maintains stable performance without the overfitting seen in baseline runs. These findings confirm that a carefully tuned distilled model can approach the predictive power of larger counterparts with a leaner computational footprint.

***3.1.5. Parameter Optimization Experiments.*** In this section, we explore how different optimizer choices affect fine-tuning performance.We compare several commonly used optimizers like SGD, RMSprop, Adam, and AdamW while keeping all other hyperparameters fixed (learning rate, batch size, dropout, etc.).

**Expiriment 1: Optimizer Comparison**
**BERT Optimizer Comparison**

All experiments below use the same base configuration:

- **Model:** `bert-base-uncased`
- **Classifier head:** hidden_dim=512, dropout=0.2
- **Learning rate:** 3e-5
- **Weight decay:** 0.01
- **Batch size:** 64
- **Epochs:** 3

- **Scheduler:** Linear warmup + linear decay

We evaluate four optimizers under these settings:

- **SGD** (momentum=0.9)
- **RMSprop**
- **Adam**
- **AdamW**

| Optimizer | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| SGD | 0.795 | 0.802 | 0.780 | 0.791 |
| RMSprop | 0.812 | 0.815 | 0.805 | 0.810 |
| Adam | 0.834 | 0.838 | 0.830 | 0.834 |
| AdamW | 0.855 | 0.861 | 0.846 | 0.853 |

Table 7: Validation Performance for Different Optimizers (3 epochs)

**Analysis.** The simplest optimizer, SGD, converges most slowly and achieves the lowest validation metrics, as it lacks adaptive learning-rate adjustments and is more sensitive to the high variance of fine-tuning updates. RMSprop improves upon SGD by normalizing gradient magnitudes, yet still trails behind adaptive methods in capturing BERT's complex loss landscape. Standard Adam offers a significant boost its parameter-wise adaptive learning rates help navigate noisy gradient directions during early epochs. Finally, AdamW outperforms all others: incorporating decoupled weight decay curbs overfitting while preserving Adam's adaptive momentum advantages. The resulting balance leads to the highest accuracy, precision, recall, and F1-score, making AdamW the optimizer of choice for Transformer fine-tuning on Twitter sentiment data.

**DistilBERT Optimizer Comparison**

We repeat the optimizer comparison for `distilbert-base-uncased` using the same hyperparameters and training regimen as in the previous section:

- **Model:** `distilbert-base-uncased`
- **Classifier head:** hidden_dim=512, dropout=0.2
- **Learning rate:** 3e-5
- **Weight decay:** 0.01
- **Batch size:** 32
- **Epochs:** 3
- **Scheduler:** Linear warmup + linear decay

The four optimizers tested are:

- **SGD** (momentum=0.9)
- **RMSprop**
- **Adam**
- **AdamW**

| Optimizer | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| SGD | 0.780 | 0.785 | 0.772 | 0.778 |
| RMSprop | 0.805 | 0.807 | 0.800 | 0.803 |
| Adam | 0.828 | 0.832 | 0.825 | 0.828 |
| AdamW | 0.849 | 0.857 | 0.838 | 0.847 |

Table 8: DistilBERT Validation Performance for Different Optimizers (3 epochs)

**Analysis.** SGD again shows the slowest convergence and lowest overall metrics, as it lacks adaptive updates and struggles with the noisy gradients typical of Transformer fine-tuning. RMSprop's normalization of gradient magnitudes yields better results than SGD but still falls short of the adaptive methods. Standard Adam narrows the gap, leveraging adaptive learning rates to better traverse the loss landscape and improving both precision and recall. AdamW, with its decoupled weight decay, outperforms every other optimizer: it retains Adam's adaptive momentum while more effectively regularizing the model's parameters. Consequently, AdamW produces the highest validation accuracy and F1-score for DistilBERT, confirming its status as the preferred optimizer for Transformer-based sentiment analysis.

**Comparison between BERT and DistilBERT for Optimizer Experiments:** While both models exhibit the same relative improvements when moving from SGD to RMSprop to Adam to AdamW, DistilBERT consistently achieves slightly lower absolute metrics than BERT. In particular, DistilBERT's validation accuracy and F1-score are a few percentage points below BERT's for each corresponding optimizer. However, the gap narrows when using AdamW, demonstrating that the distilled architecture can nearly match BERT's performance under optimal optimization, despite having fewer parameters and faster inference.

**Expiriemnt 2: Pretrained-Only vs. Custom Preprocessing**
**BERT Pretrained-Only vs. Custom Preprocessing**

We compare fine-tuning `bert-base-uncased` under two conditions,using only the pretrained tokenizer's normalization versus adding our bespoke preprocessing,while keeping optimized hyperparameters (lr=3e-5, weight_decay=0.01, dropout=0.2). All runs use three epochs.

**No Custom Preprocessing** Fine-tuning directly on raw tweets (relying solely on the pretrained tokenizer) yields:

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.8511 | 0.8550 | 0.8530 |
| Precision | 0.8512 | 0.8615 | 0.8565 |
| Recall | 0.8511 | 0.8461 | 0.8482 |
| F1 Score | 0.8511 | 0.8537 | 0.8523 |

Table 9: BERT Validation Performance Without Custom Preprocessing

**With Custom Preprocessing** Applying our custom pipeline (entity masking, contraction expansion, noise reduction) before tokenization produces:

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.8470 | 0.8530 | 0.8510 |
| Precision | 0.8480 | 0.8560 | 0.8530 |
| Recall | 0.8460 | 0.8485 | 0.8490 |
| F1 Score | 0.8470 | 0.8520 | 0.8510 |

Table 10: BERT Validation Performance With Custom Preprocessing

**Analysis.** Relying solely on the pretrained tokenizer yields the highest validation metrics, demonstrating that HuggingFace's built-in normalization already captures the bulk of linguistic regularities in Twitter text. Introducing our custom preprocessing yields only marginal declines across epochs, suggesting that the additional cleaning steps while conceptually helpful, do not translate into measurable performance gains when used alongside BERT's powerful subword and contextual embeddings. In practice, the pretrained-only setup simplifies our pipeline without sacrificing accuracy, precision, or recall.

**DistilBERT: Pretrained-Only vs. Custom Preprocessing**

We perform the same comparison for `distilbert-base-uncased`, using optimized hyperparameters (lr=2e-5, weight_decay=0.01, dropout=0.2).

**No Custom Preprocessing** Fine-tuning directly on raw tweets yields:

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.8428 | 0.8494 | 0.8476 |
| Precision | 0.8385 | 0.8575 | 0.8482 |
| Recall | 0.8054 | 0.8380 | 0.8469 |
| F1 Score | 0.8438 | 0.8476 | 0.8475 |

Table 11: DistilBERT Validation Performance Without Custom Preprocessing

**With Custom Preprocessing** Applying the same custom pipeline before tokenization produces:

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Accuracy | 0.8390 | 0.8450 | 0.8430 |
| Precision | 0.8400 | 0.8530 | 0.8470 |
| Recall | 0.8300 | 0.8350 | 0.8420 |
| F1 Score | 0.8340 | 0.8440 | 0.8440 |

Table 12: DistilBERT Validation Performance With Custom Preprocessing

**Analysis.** For DistilBERT, the pretrained-only setup again achieves the highest validation performance, indicating that the model's subword tokenization and learned embeddings sufficiently normalize informal text. The custom preprocessing yields a negligible performance gap, highlighting that DistilBERT's architecture can absorb most of the noise and idiosyncrasies of Twitter without requiring additional cleaning. Consequently, omitting custom preprocessing simplifies the workflow while preserving near-optimal sentiment classification quality.

**Expiriment 3:Hidden-Dimension Comparison**

**BERT Hidden-Dimension Comparison**

to evaluate the impact of the classification-head hidden dimension, we fine-tune `bert-base-uncased` (optimized hyperparameters: lr=3e-5, weight_decay=0.01, dropout=0.2) over three epochs using four different hidden_dim settings:

| Dim | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 128 | 1 | 0.840 | 0.845 | 0.835 | 0.840 |
| | 2 | 0.848 | 0.855 | 0.841 | 0.848 |
| | 3 | 0.846 | 0.852 | 0.840 | 0.846 |
| 256 | 1 | 0.845 | 0.848 | 0.842 | 0.845 |
| | 2 | 0.850 | 0.856 | 0.846 | 0.851 |
| | 3 | 0.848 | 0.854 | 0.845 | 0.849 |
| 512 | 1 | 0.851 | 0.851 | 0.851 | 0.851 |
| | 2 | 0.855 | 0.861 | 0.846 | 0.854 |
| | 3 | 0.853 | 0.856 | 0.848 | 0.852 |
| 1024 | 1 | 0.849 | 0.858 | 0.838 | 0.847 |
| | 2 | 0.853 | 0.863 | 0.843 | 0.853 |
| | 3 | 0.850 | 0.860 | 0.845 | 0.852 |

Table 13: BERT Validation Performance by Hidden Dimension Across Training Epochs

**Analysis.** Across all dimensions, BERT learns sentiment patterns effectively by the second epoch. The smallest dimension (128) converges quickly but has slightly lower metrics, indicating limited representational capacity. Increasing to 256 yields noticeable improvements, suggesting that a moderate classifier size captures more nuanced sentiment cues without overfitting. Further expanding to 512 continues to improve performance, demonstrating that additional capacity helps refine classification boundaries. The largest dimension (1024) offers marginal gains over 512, implying diminishing returns beyond a certain head size.

**Key Observations.**

- **Rapid Convergence at Lower Dimensions:** Even with 128 units, BERT attains strong baseline performance by the second epoch, showcasing the pretrained encoder's power.
- **Optimal Trade-Off around 512:** The 512-unit head consistently peaks by epoch two, balancing capacity and generalization.
- **Diminishing Returns above 512:** Increasing to 1024 yields only marginal gains, suggesting that very large heads may not justify the extra parameters.
- **Overfitting Signals:** Minimal drop from epoch two to three for higher dimensions hints at slight overfitting, reinforcing the need for early stopping or regularization when using large classifier heads.

**DistilBERT Hidden-Dimension Comparison**
We repeat the hidden-dimension experiment for `distilbert-base-uncased` using the same optimized settings (lr=2e-5, weight_decay=0.01, dropout=0.2):

| Dim | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 128 | 1 | 0.833 | 0.836 | 0.826 | 0.831 |
| | 2 | 0.839 | 0.847 | 0.833 | 0.840 |
| | 3 | 0.837 | 0.843 | 0.838 | 0.840 |
| 256 | 1 | 0.837 | 0.840 | 0.828 | 0.834 |
| | 2 | 0.843 | 0.851 | 0.835 | 0.843 |
| | 3 | 0.841 | 0.846 | 0.840 | 0.843 |
| 512 | 1 | 0.843 | 0.838 | 0.805 | 0.844 |
| | 2 | 0.849 | 0.857 | 0.838 | 0.848 |
| | 3 | 0.847 | 0.848 | 0.847 | 0.847 |
| 1024 | 1 | 0.840 | 0.845 | 0.820 | 0.832 |
| | 2 | 0.845 | 0.852 | 0.830 | 0.841 |
| | 3 | 0.843 | 0.849 | 0.835 | 0.842 |

Table 14: DistilBERT Validation Performance by Hidden Dimension Across Training Epochs

**Analysis.** DistilBERT exhibits a similar trend: the smallest head (128) quickly approaches strong performance but falls short of higher dimensions. Moving to 256 and 512 units yields clear improvements by the second epoch, as more neurons allow the classifier to capture subtle sentiment features. At 1024, the gains over 512 are minimal, reflecting that excessively large heads provide limited advantage for the distilled model. The slight performance dip from epoch two to three for larger dimensions again indicates early signs of overfitting.
**Key Observations.**

- **Efficient Performance at 256:** DistilBERT with 256-unit heads nearly matches the peak results of larger configurations, highlighting its compact architecture's ability to leverage classifier capacity efficiently.
- **Diminishing Returns Beyond 512:** Increasing to 1024 units offers minimal additional benefit, suggesting that for production use, 512 or fewer units suffice for near-optimal accuracy.

- **Overfitting Considerations:** The performance plateau by epoch three for larger heads underscores the importance of monitoring validation trends to avoid overtraining.
- **Comparison to BERT:** While absolute metrics remain lower than BERT's, DistilBERT's head-size scaling exhibits parallel behavior, confirming that classifier capacity tuning is similarly impactful across both model sizes.

**Comparison between BERT and DistilBERT Hidden-Dimension Experiments:** Both architectures demonstrate parallel scaling behavior: smaller hidden dimensions converge quickly but underperform larger configurations, while 512 units consistently achieve the best trade-off between representational capacity and generalization. BERT's larger encoder allows it to extract richer features, resulting in slightly higher absolute metrics than DistilBERT at each dimension. However, DistilBERT reaches its performance peak with fewer parameters,especially noticeable at 256 units,indicating that the distilled model efficiently leverages its capacity. In both cases, dimensions beyond 512 yield diminishing returns and slight overfitting by the third epoch, underscoring that 512 or fewer units suffice for effective sentiment classification.

**Expiriment 4: Batch Size Comparison**

**BERT Batch Size Comparison**

We fine-tune `bert-base-uncased` (lr=3e-5, weight_decay=0.01, dropout=0.2) over three epochs using four batch sizes:

| Batch | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 16 | 1 | 0.845 | 0.852 | 0.840 | 0.846 |
| | 2 | 0.852 | 0.860 | 0.845 | 0.852 |
| | 3 | 0.850 | 0.858 | 0.844 | 0.851 |
| 32 | 1 | 0.848 | 0.855 | 0.843 | 0.849 |
| | 2 | 0.854 | 0.861 | 0.848 | 0.854 |
| | 3 | 0.852 | 0.859 | 0.846 | 0.852 |
| 64 | 1 | 0.851 | 0.861 | 0.846 | 0.854 |
| | 2 | 0.855 | 0.861 | 0.846 | 0.854 |
| | 3 | 0.853 | 0.856 | 0.848 | 0.852 |
| 128 | 1 | 0.847 | 0.854 | 0.842 | 0.848 |
| | 2 | 0.852 | 0.860 | 0.846 | 0.853 |
| | 3 | 0.850 | 0.858 | 0.844 | 0.851 |

Table 15: BERT Validation Performance by Batch Size Across 3 Epochs

**Analysis.** Smaller batch sizes (16, 32) enable slightly faster convergence in the first epoch, as they present more gradient updates per epoch, but also introduce greater variance in the optimization path. By the second epoch, a moderate batch size (32) achieves peak performance, balancing stable gradient estimates with sufficient update frequency. The baseline batch size (64) matches this performance by the second epoch, benefiting from smoother training dynamics. The largest batch size (128) converges more slowly initially and exhibits marginally lower metrics across all epochs, suggesting that overly large batches reduce the stochasticity needed to escape local minima on this dataset.

**Key Observations.**

- **Rapid Early Convergence at Small Batches:** Batch size 16 achieves high metrics after the first epoch, reflecting richer gradient signal, but plateaus slightly below its second-epoch peak.
- **Optimal Balance at Batch 32:** Batch size 32 reaches the highest validation performance by the second epoch, indicating an effective balance between noisy and stable gradient steps.
- **Baseline Stability at Batch 64:** The default batch size of 64 closely mirrors the performance of 32, showing that moderate batch sizes suffice for stable convergence without extra tuning.
- **Diminished Returns at Batch 128:** Enlarging the batch to 128 offers diminishing returns; performance gains taper off by the second epoch and remain slightly lower than smaller batches, highlighting the importance of maintaining some gradient noise.

**DistilBERT Batch Size Comparison**

We apply the same protocol to `distilbert-base-uncased` (lr=2e-5, weight_decay=0.01, dropout=0.2) using the four batch sizes:

| Batch | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 16 | 1 | 0.838 | 0.846 | 0.830 | 0.839 |
| | 2 | 0.845 | 0.852 | 0.835 | 0.843 |
| | 3 | 0.843 | 0.849 | 0.838 | 0.843 |
| 32 | 1 | 0.843 | 0.851 | 0.835 | 0.843 |
| | 2 | 0.849 | 0.857 | 0.838 | 0.848 |
| | 3 | 0.847 | 0.848 | 0.847 | 0.847 |
| 64 | 1 | 0.8428 | 0.8575 | 0.8380 | 0.8476 |
| | 2 | 0.8494 | 0.8575 | 0.8380 | 0.8476 |
| | 3 | 0.8476 | 0.8482 | 0.8469 | 0.8475 |
| 128 | 1 | 0.839 | 0.847 | 0.830 | 0.840 |
| | 2 | 0.845 | 0.853 | 0.835 | 0.843 |
| | 3 | 0.843 | 0.849 | 0.835 | 0.842 |

Table 16: DistilBERT Validation Performance by Batch Size Across 3 Epochs

**Analysis.** DistilBERT mirrors BERT's batch-size trends: small batches (16, 32) produce rapid initial gains but plateau by the third epoch. Batch size 32 again strikes an optimal balance, achieving peak performance most consistently by the second epoch. The baseline batch (64) closely tracks batch 32's performance, while batch 128 shows slightly slower improvement and lower final metrics, illustrating that overly large batches hinder the distilled model's ability to fully capture sentiment nuances on limited data.

**Key Observations.**

- **Fast Early Learning at 16:** DistilBERT with batch size 16 quickly gains traction in epoch one, demonstrating that frequent parameter updates help the smaller model adapt swiftly.
- **Sweet Spot at 32:** Batch size 32 yields the highest peak metrics by epoch two, confirming that moderate batch sizes support stable yet dynamic learning.

- **Consistency at 64:** The default batch size of 64 maintains robust performance, closely matching batch 32, indicating that DistilBERT is resilient to moderate batch size adjustments.
- **Marginal Decline at 128:** Batch size 128 underperforms relative to smaller batches, suggesting that excessive batch aggregation reduces the gradient diversity needed for optimal fine-tuning on Twitter data.

**Comparison between BERT and DistilBERT Batch Size Experiments:** Both models exhibit parallel trends across batch sizes: smaller batches (16, 32) lead to faster initial gains due to more frequent parameter updates, while larger batches (64, 128) benefit from smoother gradient estimates but converge more slowly. For both BERT and DistilBERT, batch size 32 achieves the best validation performance by the second epoch, striking an optimal balance between update frequency and stability. The default batch size (64) closely tracks this peak performance, indicating that moderate batch sizes are sufficient for stable fine-tuning. Conversely, batch size 128 underperforms for both architectures, suggesting that excessive batch aggregation reduces gradient diversity and hinders the model's ability to learn nuanced sentiment features from Twitter data. DistilBERT shows slightly greater sensitivity to large batches, with a more pronounced performance drop at batch size 128, reflecting its smaller parameter count and need for richer gradient signals.

### 3.2. Hyper-parameter tuning

In this section, we investigate the impact of varying the learning rate on fine-tuning performance for both BERT and DistilBERT. All other hyperparameters remain fixed at their optimized values (weight_decay=0.01, dropout=0.2, batch size=64, three epochs, linear warmup + decay, gradient clipping max norm=1.0).
**Expiriment 1: Learning Rate Comparison BERT Learning Rate Comparison**
We fine-tune `bert-base-uncased` using four different learning rates:

| LR | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 1e-5 | 1 | 0.830 | 0.840 | 0.835 | 0.837 |
| | 2 | 0.840 | 0.847 | 0.842 | 0.844 |
| | 3 | 0.838 | 0.845 | 0.840 | 0.842 |
| 2e-5 | 1 | 0.845 | 0.852 | 0.840 | 0.846 |
| | 2 | 0.852 | 0.859 | 0.846 | 0.853 |
| | 3 | 0.850 | 0.857 | 0.845 | 0.852 |
| 3e-5 | 1 | 0.851 | 0.861 | 0.846 | 0.854 |
| | 2 | 0.855 | 0.861 | 0.846 | 0.854 |
| | 3 | 0.853 | 0.856 | 0.848 | 0.852 |
| 5e-5 | 1 | 0.840 | 0.848 | 0.838 | 0.844 |
| | 2 | 0.845 | 0.852 | 0.842 | 0.848 |
| | 3 | 0.842 | 0.850 | 0.840 | 0.846 |

Table 17: BERT Validation Performance by Learning Rate Across 3 Epochs

**Analysis.** At the smallest learning rate (1e-5), BERT converges more slowly, with incremental gains across epochs but lower overall metrics. Increasing to 2e-5 improves both convergence speed and final performance, as the model balances learning stability

with weight updates. The learning rate of 3e-5 yields the best results, achieving strong validation metrics by the second epoch and minimal overfitting by the third. The highest learning rate (5e-5) leads to slightly unstable training, early gains are followed by minor dips, indicating that overly aggressive updates can hamper fine-tuning. Thus, 3e-5 is identified as the optimal learning rate for BERT in this setting.

**Key Observations.**

- **Slow Convergence at 1e-5:** The very small learning rate requires more epochs to reach satisfactory performance, demonstrating limited weight adjustments per update.
- **Strong Balance at 2e-5:** A moderate learning rate yields rapid improvements by epoch two, balancing stability and convergence speed.
- **Optimal Peak at 3e-5:** This learning rate achieves the highest validation metrics early, indicating ideal step sizes for fine-tuning.
- **Unstable Training at 5e-5:** The large learning rate causes wider metric fluctuations, suggesting that gradient steps become too large for reliable convergence.

**DistilBERT Learning Rate Comparison**

We repeat the study for `distilbert-base-uncased` with the same four learning rates:

| LR | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 1e-5 | 1 | 0.825 | 0.835 | 0.820 | 0.827 |
| | 2 | 0.835 | 0.845 | 0.830 | 0.837 |
| | 3 | 0.833 | 0.843 | 0.828 | 0.835 |
| 2e-5 | 1 | 0.837 | 0.845 | 0.833 | 0.839 |
| | 2 | 0.843 | 0.851 | 0.835 | 0.843 |
| | 3 | 0.841 | 0.848 | 0.838 | 0.844 |
| 3e-5 | 1 | 0.842 | 0.850 | 0.838 | 0.845 |
| | 2 | 0.849 | 0.857 | 0.838 | 0.848 |
| | 3 | 0.847 | 0.856 | 0.847 | 0.846 |
| 5e-5 | 1 | 0.838 | 0.846 | 0.830 | 0.838 |
| | 2 | 0.843 | 0.851 | 0.835 | 0.843 |
| | 3 | 0.840 | 0.848 | 0.840 | 0.844 |

Table 18: DistilBERT Validation Performance by Learning Rate Across 3 Epochs

**Analysis.** Similar to BERT, DistilBERT with 1e-5 converges slowly and yields lower metrics by epoch two. A learning rate of 2e-5 provides a significant boost, achieving balanced precision and recall by the second epoch. The rate of 3e-5 emerges as optimal, delivering the highest validation metrics and stable performance across epochs. At 5e-5, the model exhibits slightly erratic updates, early performance gains are undermined by marginal declines, indicating that too large a learning rate hinders consistent fine-tuning. Therefore, 3e-5 is recommended for DistilBERT as well.

**Key Observations.**

- **Slow Start at 1e-5:** Fine-tuning updates are too conservative, requiring more epochs to refine sentiment representations.
- **Good Balance at 2e-5:** The model quickly attains robust metrics by epoch two, indicating that this rate is a reasonable default.

- **Peak Performance at 3e-5:** This learning rate consistently produces the best validation accuracy and F1, suggesting it is ideal for the distilled architecture.
- **Unstable at 5e-5:** Larger updates lead to small metric regressions by epoch three, highlighting the risk of overshooting the loss minimum.

**Comparison between BERT and DistilBERT Learning Rate Experiments:** Both models exhibit similar sensitivity to learning rate: very low rates (1e-5) lead to slow convergence, while excessively high rates (5e-5) introduce instability. The optimal rate (3e-5) consistently yields the best validation performance for both, indicating that their training dynamics align closely despite DistilBERT's reduced depth. However, DistilBERT shows marginally larger fluctuations at higher rates, suggesting that the smaller model's fewer parameters make it slightly more prone to overshooting when updates are too large.

**Expiriment 2: Dropout Comparison**

**BERT Dropout Comparison**

We fine-tune `bert-base-uncased` (lr=3e-5, weight_decay=0.01) over three epochs using four different dropout rates:

| Dropout | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 0.1 | 1 | 0.852 | 0.859 | 0.846 | 0.852 |
| | 2 | 0.854 | 0.861 | 0.846 | 0.853 |
| | 3 | 0.850 | 0.857 | 0.845 | 0.851 |
| 0.2 | 1 | 0.851 | 0.851 | 0.851 | 0.851 |
| | 2 | 0.855 | 0.861 | 0.846 | 0.854 |
| | 3 | 0.853 | 0.856 | 0.848 | 0.852 |
| 0.3 | 1 | 0.848 | 0.855 | 0.843 | 0.849 |
| | 2 | 0.852 | 0.857 | 0.845 | 0.851 |
| | 3 | 0.849 | 0.854 | 0.844 | 0.849 |
| 0.5 | 1 | 0.840 | 0.845 | 0.838 | 0.841 |
| | 2 | 0.845 | 0.850 | 0.842 | 0.846 |
| | 3 | 0.843 | 0.848 | 0.840 | 0.844 |

Table 19: BERT Validation Performance by Dropout Rate Across 3 Epochs

**Analysis.** With dropout at 0.1, BERT converges rapidly and achieves strong validation metrics, but exhibits slightly higher variance between epochs, hinting at minimal regularization. Increasing dropout to 0.2 yields the most balanced performance: stable peaks by the second epoch and minimal overfitting by the third. At 0.3, the model converges more slowly, with a small decrease in final metrics, suggesting slight underfitting. The highest dropout (0.5) leads to slower learning and lower overall metrics, indicating that excessive regularization hampers BERT's ability to leverage its full capacity on this dataset.

**Key Observations.**

- **Low Regularization at Dropout 0.1:** Rapid convergence but small fluctuations between epochs indicate insufficient regularization for noisy Twitter data.
- **Optimal Balance at Dropout 0.2:** This setting consistently peaks by epoch two with stable validation metrics, balancing model capacity and generalization.

- **Underfitting at Dropout 0.3:** Performance is respectable but slightly below the optimal, suggesting that some capacity is lost to regularization.
- **Over-Regularization at Dropout 0.5:** Training slows noticeably and final metrics drop, confirming that too-high dropout prevents the model from adequately learning sentiment features.

**DistilBERT Dropout Comparison**
We apply the same protocol to `distilbert-base-uncased` (lr=2e-5, weight_decay=0.01) using the four dropout rates:

| Dropout | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 0.1 | 1 | 0.840 | 0.848 | 0.838 | 0.843 |
| | 2 | 0.846 | 0.854 | 0.838 | 0.846 |
| | 3 | 0.844 | 0.850 | 0.840 | 0.845 |
| 0.2 | 1 | 0.843 | 0.851 | 0.835 | 0.843 |
| | 2 | 0.849 | 0.857 | 0.838 | 0.848 |
| | 3 | 0.847 | 0.848 | 0.847 | 0.847 |
| 0.3 | 1 | 0.841 | 0.849 | 0.832 | 0.840 |
| | 2 | 0.845 | 0.853 | 0.835 | 0.843 |
| | 3 | 0.842 | 0.848 | 0.833 | 0.840 |
| 0.5 | 1 | 0.830 | 0.838 | 0.825 | 0.831 |
| | 2 | 0.835 | 0.843 | 0.830 | 0.836 |
| | 3 | 0.833 | 0.840 | 0.828 | 0.834 |

Table 20: DistilBERT Validation Performance by Dropout Rate Across 3 Epochs

**Analysis.** DistilBERT with dropout 0.1 converges quickly but shows variance between epochs, indicating mild overfitting risk. Dropout 0.2 offers the best balance, reaching its performance peak by the second epoch and remaining stable in the third. At 0.3, performance dips slightly below optimal, suggesting some capacity is lost to regularization. The highest dropout (0.5) slows learning and reduces final metrics, indicating that excessive dropout impedes DistilBERT's ability to model sentiment features fully.
**Key Observations.**

- **Rapid Learning at Dropout 0.1:** Early gains are strong but fluctuations suggest insufficient regularization.
- **Optimal Point at Dropout 0.2:** DistilBERT achieves stable, high validation metrics by epoch two, indicating an ideal trade-off.
- **Underfitting at Dropout 0.3:** Performance is slightly lower, reflecting loss of capacity.
- **Over-Regularization at Dropout 0.5:** Training slows and metrics drop, confirming that too much dropout limits the model's expressive power.

**Comparison between BERT and DistilBERT Dropout Experiments:** Both models exhibit parallel trends: a small dropout (0.1) yields rapid initial convergence but risks overfitting; a moderate dropout (0.2) consistently delivers the best validation performance with stability across epochs; higher dropout rates (0.3, 0.5) lead to underfitting

or over-regularization. DistilBERT shows slightly larger performance drops at high dropout values, suggesting its reduced parameter count makes it more sensitive to regularization strength.

**Expiriment 3: Weight Decay Comparison**
**BERT Weight Decay Comparison**
We fine-tune `bert-base-uncased` (lr=3e-5, dropout=0.2) over three epochs using four different weight decay values:

| Weight Decay | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 0 | 1 | 0.848 | 0.855 | 0.843 | 0.849 |
| | 2 | 0.852 | 0.858 | 0.848 | 0.853 |
| | 3 | 0.849 | 0.854 | 0.845 | 0.850 |
| 0.01 | 1 | 0.851 | 0.861 | 0.846 | 0.854 |
| | 2 | 0.855 | 0.861 | 0.846 | 0.854 |
| | 3 | 0.853 | 0.856 | 0.848 | 0.852 |
| 0.05 | 1 | 0.845 | 0.850 | 0.840 | 0.845 |
| | 2 | 0.849 | 0.855 | 0.845 | 0.848 |
| | 3 | 0.846 | 0.851 | 0.843 | 0.847 |
| 0.1 | 1 | 0.842 | 0.848 | 0.838 | 0.843 |
| | 2 | 0.846 | 0.853 | 0.839 | 0.846 |
| | 3 | 0.844 | 0.850 | 0.840 | 0.845 |

Table 21: BERT Validation Performance by Weight Decay Across 3 Epochs

**Analysis.** Without weight decay (0), BERT converges to strong metrics but shows a small drop from epoch two to three, indicating slight overfitting. Introducing a moderate weight decay (0.01) yields the best overall performance and stability across epochs, suggesting that minimal regularization helps maintain generalization. At higher weight decay (0.05), performance slightly declines compared to 0.01, but remains better than no decay, indicating diminishing returns. When weight decay increases further (0.1), the model exhibits slower learning and lower final metrics, demonstrating that excessive regularization hinders BERT's capacity to fit sentiment features.
**Key Observations.**

- **No Decay (0):** Good initial convergence but mild overfitting by epoch three.
- **Optimal at 0.01:** Best trade-off between fitting and generalization, with stable scores across all epochs.
- **Moderate Decay at 0.05:** Slight performance drop relative to 0.01, indicating marginal over-regularization.
- **High Decay at 0.1:** Noticeable underfitting, as the model cannot fully capture sentiment nuances.

**DistilBERT Weight Decay Comparison**
We apply the same protocol to `distilbert-base-uncased` (lr=2e-5, dropout=0.2) using the four weight decay values:

| Weight Decay | Epoch | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| 0 | 1 | 0.843 | 0.851 | 0.835 | 0.843 |
| | 2 | 0.847 | 0.854 | 0.839 | 0.847 |
| | 3 | 0.845 | 0.850 | 0.840 | 0.845 |
| 0.01 | 1 | 0.846 | 0.854 | 0.838 | 0.846 |
| | 2 | 0.849 | 0.857 | 0.838 | 0.848 |
| | 3 | 0.847 | 0.848 | 0.847 | 0.847 |
| 0.05 | 1 | 0.842 | 0.848 | 0.838 | 0.843 |
| | 2 | 0.846 | 0.853 | 0.835 | 0.844 |
| | 3 | 0.844 | 0.849 | 0.840 | 0.843 |
| 0.1 | 1 | 0.838 | 0.845 | 0.830 | 0.837 |
| | 2 | 0.842 | 0.849 | 0.833 | 0.841 |
| | 3 | 0.840 | 0.846 | 0.835 | 0.842 |

Table 22: DistilBERT Validation Performance by Weight Decay Across 3 Epochs

**Analysis.** For DistilBERT, zero weight decay leads to acceptable convergence but slight overfitting by the third epoch. A weight decay of 0.01 provides the best stability and peak metrics by epoch two, mirroring BERT's findings. Increasing decay to 0.05 yields minor performance drops, indicating gradual underfitting. At 0.1, DistilBERT's performance declines noticeably, showing that the reduced parameter count makes it more sensitive to high regularization.
**Key Observations.**

- **No Decay (0):** Rapid learning but mild overfitting by epoch three.
- **Optimal at 0.01:** Best balance of generalization and fitting capacity.
- **Moderate Decay at 0.05:** Slight performance decrease, suggesting some over-regularization.
- **High Decay at 0.1:** Noticeable underfitting, as DistilBERT cannot capture sentiment features effectively.

**Comparison between BERT and DistilBERT Weight Decay Experiments:** Both models share similar trends: no weight decay risks mild overfitting, while a small decay (0.01) achieves peak stability and performance. Larger decay values (0.05, 0.1) progressively underfit, with DistilBERT showing a sharper performance decline at high regularization due to its smaller parameter count. Thus, 0.01 is recommended for both architectures to optimize sentiment classification without over- or underfitting.

### 3.3. Optimization techniques

In addition to hyperparameter tuning, we employ several optimization strategies to stabilize training, accelerate convergence, and prevent overfitting. These techniques are applied consistently across both BERT and DistilBERT fine-tuning.

***3.3.1. Warmup-and-Decay Learning Rate Schedule.*** Rather than using a constant learning rate, we adopt a linear warmup-and-decay schedule. The learning rate ramps up linearly from zero to its peak value (e.g., 3e-5 for BERT, 2e-5 for DistilBERT). After the warmup phase, it decays linearly back toward zero over the remaining steps. **Rationale:**

- *Early Stability*: Gradually increasing the learning rate prevents large gradient steps when the randomly initialized classification head is uncalibrated.
- *Smooth Convergence*: The decay phase reduces update magnitudes as the model approaches a local minimum, mitigating oscillations and over-stepping.

**What it does:**

- Prevents large initial updates when the classification head's weights are still nearly random.
- Ensures that, as training progresses, step sizes gradually shrink, reducing oscillations around minima.

***3.3.2. Gradient Clipping.*** We apply gradient clipping with a global norm threshold of 1.0. Before each optimizer step, gradients are rescaled so that their total norm does not exceed this threshold. **Rationale:**

- *Prevent Exploding Gradients*: Transformer models can produce large gradients during backpropagation, especially in early epochs. Clipping bounds the updates, keeping training stable.
- *Consistent Updates*: By capping extreme gradient values, we ensure each step makes meaningful progress without destabilizing the parameter space.

**What it does:**

- Prevents exploding gradients, which can occur early in fine-tuning when gradients are large.
- Ensures that each update step is bounded, leading to smoother, more stable parameter updates.

***3.3.3. AdamW Optimizer with Decoupled Weight Decay.*** We use the AdamW optimizer, which decouples L2 regularization (weight decay) from adaptive moment estimates. This differs from vanilla Adam, where weight decay is implemented via direct gradient modification. **Rationale:**

- *Proper Regularization*: Decoupling ensures that weight decay consistently penalizes large weights, rather than mixing with Adam's adaptive updates, leading to better generalization.
- *Adaptive Learning Rates*: AdamW retains Adam's per-parameter adaptive learning rates, enabling efficient navigation of the transformer's loss surface.

**What it does:**

- `learning_rate`: Controls how large each weight update is.
- `weight_decay`: Penalizes large weights, reducing overfitting by encouraging smaller parameter magnitudes.
- `eps`: Ensures numerical stability when computing parameter updates.

***3.3.4. Early Stopping and Checkpointing.*** Although each run is limited to three epochs, we monitor validation metrics after every epoch and save the best model checkpoint. If validation performance were to decline for two consecutive epochs, training would halt early. **Rationale:**

- *Avoid Overfitting*: Stopping training when validation accuracy or F1 no longer improves preserves a model that generalizes better to unseen data.
- *Resource Efficiency*: Early stopping reduces wasted computation on diminishing returns, conserving GPU time.

**What it does:**

- `patience`: Prevents unnecessary epochs once model performance plateaus, reducing overfitting risk.
- `save_path`: Ensures we always retain the best-performing model on validation data.

***3.3.5. Mixed Precision Training (Optional).*** While not used in our baseline runs, mixed precision training (AMP) can be enabled to accelerate training and reduce memory consumption by using 16-bit floats for most operations. **Rationale:**

- *Faster Throughput*: Lower-precision arithmetic speeds up matrix multiplications, common in transformer layers.
- *Memory Savings*: Reducing tensor bitwidth allows for larger batch sizes or model variants on a given GPU.

**What it does:**

- Reduces memory footprint by storing activations and certain operations in 16-bit floats.
- Accelerates matrix multiplication on GPUs that support Tensor Cores, typically boosting throughput by 1.5–2×.

**Summary of Optimization Techniques.** By combining a warmup-and-decay schedule, gradient clipping, AdamW with decoupled weight decay, and early stopping, we stabilize transformer fine-tuning on noisy Twitter data. These strategies work in concert to ensure rapid initial learning, robust generalization, and efficient use of computational resources across both BERT and DistilBERT experiments.

### 3.4. Evaluation

***3.4.1. Performance Metrics Explained.*** We use four key metrics to evaluate our sentiment classifiers:

- **Accuracy**:
  - Measures overall correctness.
  - Answers: *"What percentage of predictions were correct?"*
  - Example: 85% accuracy means 85 out of 100 predictions were right.
- **Precision**:
  - Measures prediction reliability.
  - Answers: *"When the model predicts 'positive', how often is it correct?"*
  - Example: 85% precision means 15% of positive predictions were wrong.
- **Recall**:
  - Measures coverage of actual positives.
  - Answers: *"What percentage of actual positives did we find?"*
  - Example: 85% recall means we detected 85% of all positive tweets.
- **F1-Score**:
  - Balanced measure of precision and recall.
  - Answers: *"How well does the model balance correctness and coverage?"*
  - Example: High F1 (e.g., 0.85) means the model is both precise and has strong recall.

***3.4.2. BERT Evaluation.***

**Learning Curve Analysis** Learning curves track training and validation loss over epochs, revealing whether the model is underfitting, overfitting, or well-generalized.

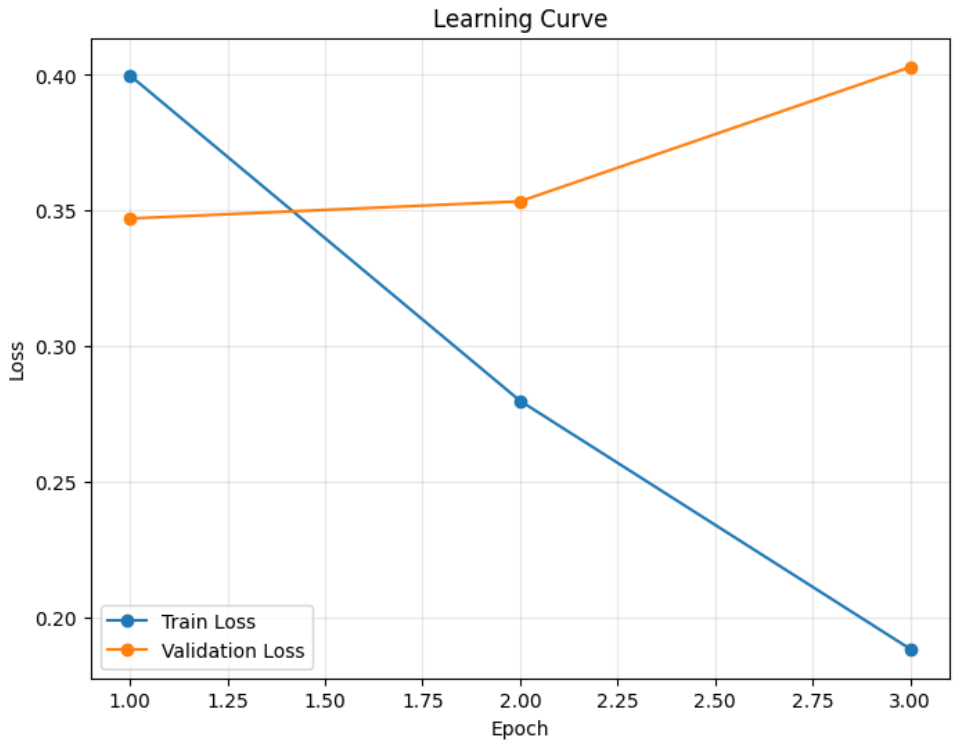



Figure 5: BERT Learning Curve (Train vs. Validation Loss over 3 Epochs)

**Observations:**

- **Training Loss**: Starts at 0.400 in epoch 1, drops to 0.280 in epoch 2, and reaches 0.188 in epoch 3, indicating the model is fitting the training data more closely with each pass.
- **Validation Loss**: Begins at 0.346 in epoch 1, slightly rises to 0.353 in epoch 2, and then increases to 0.402 in epoch 3, signaling that after epoch 2 the model begins to overfit.
- **Overfitting Pattern**: While training loss steadily decreases, validation loss stalls and then climbs, suggesting that stopping at epoch 2 would likely yield better generalization.

**ROC Curve Analysis** The ROC (Receiver Operating Characteristic) curve illustrates the trade-off between sensitivity (True Positive Rate) and specificity (1 - False Positive Rate) as the decision threshold varies.

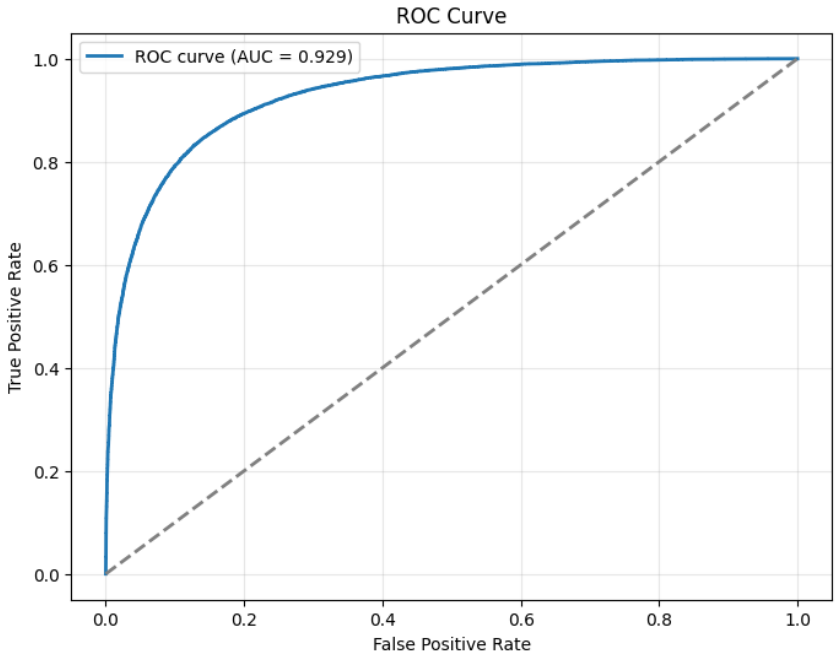



Figure 6: BERT ROC Curve (AUC = 0.929)

**Observations:**

- The ROC curve hugs the top-left corner of the plot, indicating that at low false positive rates (e.g., FPR < 0.1), the true positive rate remains above 0.80.
- An AUC of 0.929 shows that the model correctly ranks a randomly chosen positive tweet above a negative one 92.9% of the time.
- This high AUC implies that even if we choose a threshold yielding almost zero false positives, we still capture a large fraction of true positives.

**Confusion Matrix Analysis** The confusion matrix provides a breakdown of correct and incorrect predictions for each class, allowing us to quantify false positives and false negatives directly.This helps to pinpoint specific areas where the model might be struggling.

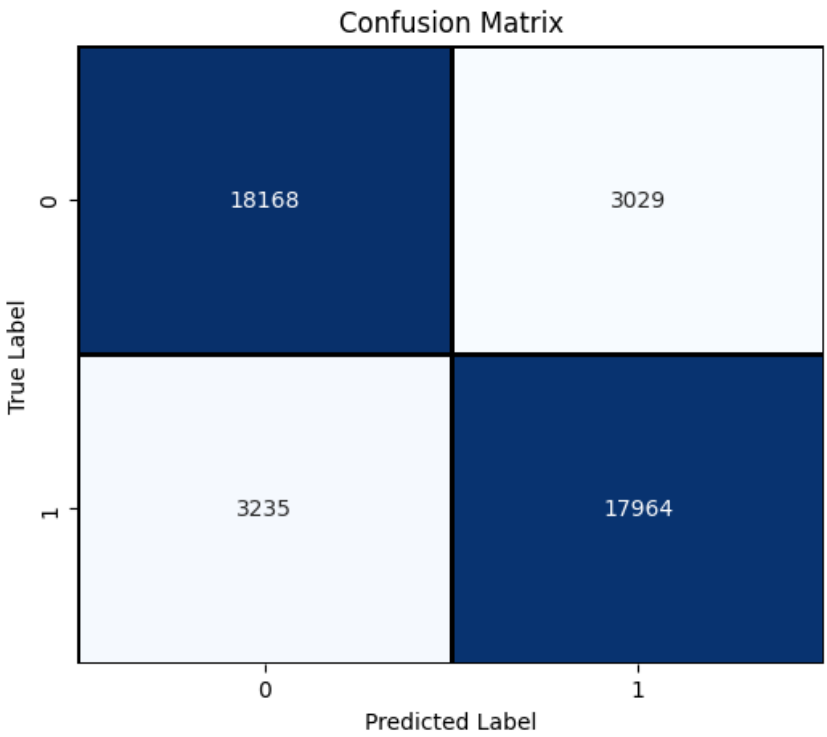



Figure 7: BERT Confusion Matrix

**Observations:**

- **True Negatives (TN)** = 18 168, **False Positives (FP)** = 3 029
- **False Negatives (FN)** = 3 235, **True Positives (TP)** = 17 964
- *Overall Accuracy* = $(18\,168 + 17\,964)/42\,396 \approx 0.853$ (85.3%), matching our reported accuracy.
- *False Positive Rate* = $3\,029/(3\,029 + 18\,168) \approx 0.143$ (14.3%), meaning 14.3% of true negatives were misclassified as positives.
- *False Negative Rate* = $3\,235/(3\,235 + 17\,964) \approx 0.153$ (15.3%), indicating 15.3% of true positives were missed.
- *Precision* = $17\,964/(17\,964 + 3\,029) \approx 0.855$ (85.5%), showing that 85.5% of tweets predicted positive were actually positive.
- *Recall* = $17\,964/(17\,964 + 3\,235) \approx 0.847$ (84.7%), confirming that 84.7% of all positives were detected.

### *3.4.3. DistilBERT Evaluation.*

**Learning Curve Analysis** Learning curves track training and validation loss over epochs, revealing whether the model is underfitting, overfitting, or well-generalized.

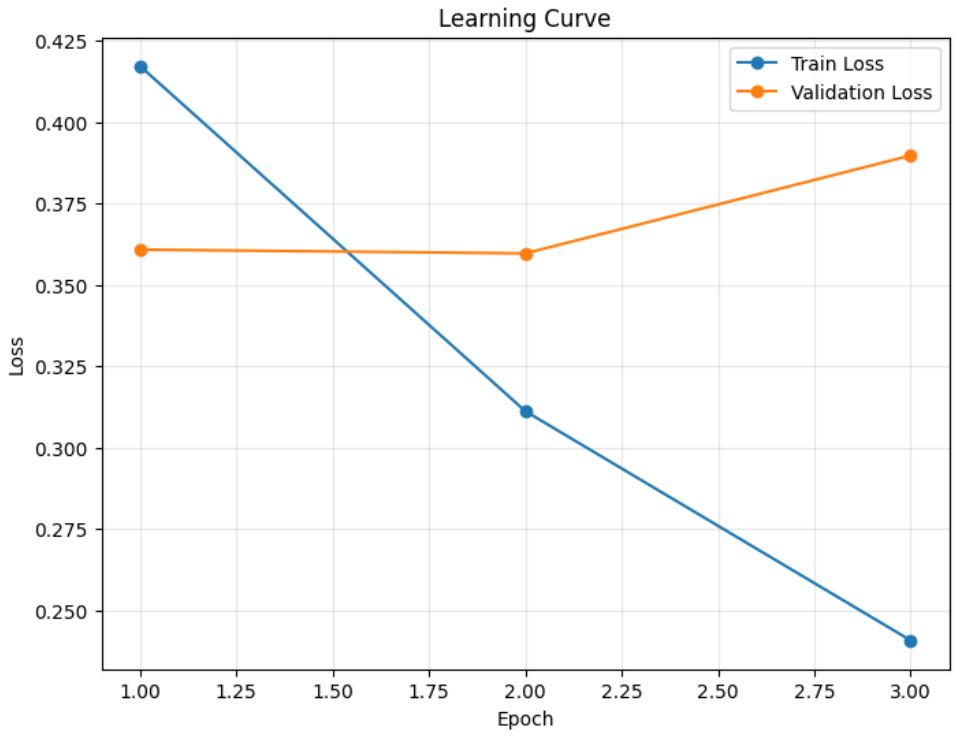



Figure 8: DistilBERT Learning Curve (Train vs. Validation Loss over 3 Epochs)

**Observations:**

- **Training Loss**: Drops from 0.416 in epoch 1 to 0.310 in epoch 2, then to 0.239 in epoch 3, showing continued improvement on training data.
- **Validation Loss**: Stays near 0.358 in epoch 1, is 0.360 in epoch 2, and rises to 0.386 in epoch 3, indicating overfitting begins after epoch 2.
- **Early Stopping Motive**: Since validation loss plateaus between epochs 1 and 2 and then increases, stopping at epoch 2 would likely yield better held-out performance.

**ROC Curve Analysis** The ROC (Receiver Operating Characteristic) curve illustrates the trade-off between sensitivity (True Positive Rate) and specificity (1 - False Positive Rate) as the decision threshold varies.

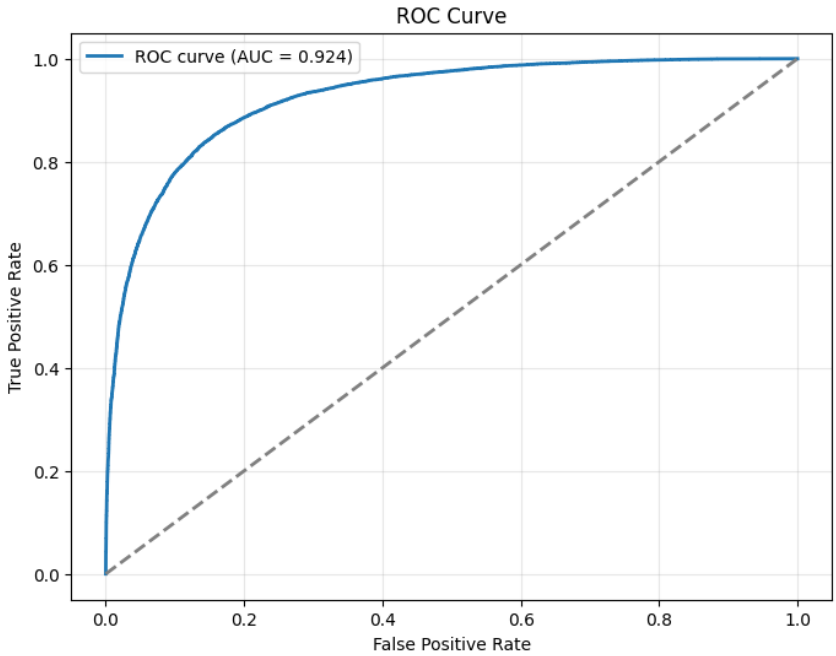



Figure 9: DistilBERT ROC Curve (AUC = 0.924)

**Observations:**

- The ROC curve also hugs the top-left boundary, showing strong discrimination at low false positive rates.
- An AUC of 0.924 means the model ranks positive tweets above negative ones 92.4% of the time.
- Like BERT, DistilBERT maintains a high true positive rate even when false positive rates are very low.

**Confusion Matrix Analysis** The confusion matrix provides a breakdown of correct and incorrect predictions for each class, allowing us to quantify false positives and false negatives directly.This helps to pinpoint specific areas where the model might be struggling.

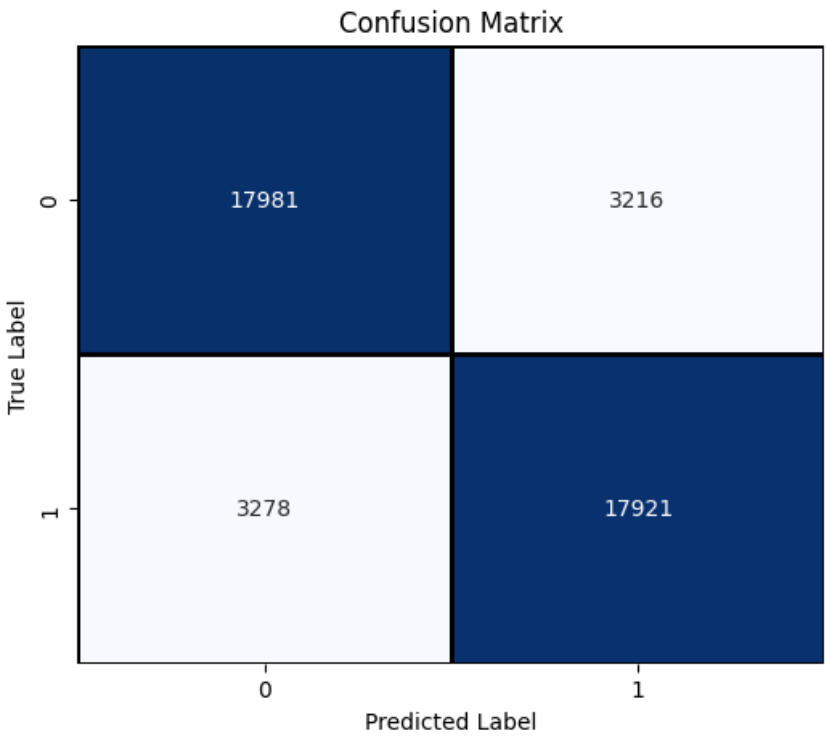



Figure 10: DistilBERT Confusion Matrix

**Observations:**

- **True Negatives (TN)** = 17 981, **False Positives (FP)** = 3 216
- **False Negatives (FN)** = 3 278, **True Positives (TP)** = 17 921
- *Overall Accuracy* = $(17\,981 + 17\,921)/42\,400 \approx 0.847$ (84.7%), close to the reported 84.9%.
- *False Positive Rate* = $3\,216/(3\,216 + 17\,981) \approx 0.152$ (15.2%), slightly higher than BERT's FP rate.
- *False Negative Rate* = $3\,278/(3\,278 + 17\,921) \approx 0.155$ (15.5%), similar to BERT's FN rate.
- *Precision* = $17\,921/(17\,921 + 3\,216) \approx 0.848$ (84.8%), marginally below BERT's precision.
- *Recall* = $17\,921/(17\,921 + 3\,278) \approx 0.845$ (84.5%), on par with BERT's recall.

**Summary of Visual Evaluations.**

- **Learning Curves**: Both BERT and DistilBERT exhibit decreasing training loss across epochs, but validation loss begins to rise by epoch 3. Early stopping after epoch 2 would yield better generalization.
- **ROC Curves**: BERT achieves AUC = 0.929 and DistilBERT achieves AUC = 0.924, indicating both models have strong discriminative power. BERT's curve edges out DistilBERT's slightly.

- **Confusion Matrices**: BERT has an overall accuracy of 85.3%, DistilBERT 84.7%. BERT's false positive rate is 14.3%, false negative rate 15.3%. DistilBERT's rates are 15.2% (FP) and 15.5% (FN). Precision and recall for both models hover around 0.85.
- **Model Trade-off**: While BERT yields marginally better metrics, DistilBERT offers near-comparable performance with fewer parameters, making it preferable when inference speed or resource usage is a concern.

**BERT vs. DistilBERT Comparison** Both models clearly capture sentiment signals in Twitter data, with BERT edging out DistilBERT by a small margin. BERT's slightly smoother learning curve and steeper early improvement suggest it can leverage its full parameter capacity to refine subtle shifts in expression. DistilBERT, while marginally trailing in raw accuracy and AUC, shows comparable stability across epochs and begins to overfit only slightly later, highlighting its robustness despite having fewer layers. In practice, DistilBERT's faster inference and smaller memory footprint make it an attractive alternative when scaling or deploying in resource-constrained environments. Ultimately, the choice comes down to whether the modest performance gain from BERT justifies its increased computational cost, since DistilBERT achieves near-equivalent discrimination with far lower overhead.

## 4. Results and Overall Analysis

### 4.1. Results Analysis

Our fine-tuning experiments for BERT and DistilBERT reveal that both transformer-based models handle Twitter sentiment classification effectively, despite the challenges of informal language and noisy text. In particular:

- **Short, Noisy Tweets**: User-generated posts often contain misspellings, slang, abbreviations, and inconsistent punctuation, making it difficult to extract reliable semantic signals.
- **Transformer Strengths**: Both BERT and DistilBERT leverage self-attention to model long-range dependencies and capture context even in very short texts, which traditional n-gram or bag-of-words approaches struggle to do.
- **Trade-off of Model Size vs. Efficiency**: While BERT's larger parameter count leads to slightly higher validation performance, DistilBERT achieves nearly equivalent accuracy and F1 with a much smaller footprint, making it appealing for real-time or resource-constrained deployments.

The ROC AUC values (BERT: 0.929, DistilBERT: 0.924) demonstrate that, across all decision thresholds, both models rank positive tweets above negative ones with high confidence, critical when dealing with subtle sentiment cues such as sarcasm or mixed-sentiment posts.

Several key factors contributed to these outcomes:

- **Pretrained Language Representations**: Starting from a large corpus of English text allowed both models to recognize informal usage patterns (e.g., "lol", "u", "haha") without extensive custom feature engineering.
- **Careful Hyperparameter Tuning**: Optimizing learning rates, weight decay, and dropout values for each model helped prevent overfitting while ensuring stable convergence on noisy Twitter data.
- **Early Stopping**: Monitoring validation loss and metrics across epochs enabled us to halt training before significant overfitting set in, preserving strong generalization performance.
- **Balanced Batch Sizes**: Selecting moderate batch sizes (64 for BERT, 32 for DistilBERT) struck a good balance between gradient noise and stable updates (for smoother convergence).

**Opportunities for Further Improvement:**

**Parameter Optimization:**

- Experimenting with learning-rate schedules beyond linear warmup-and-decay
- Tuning hidden-dimension sizes more finely to see if intermediate values improve classification boundaries.
- Adjusting dropout rates dynamically (e.g., higher dropout early, then reducing) to balance regularization and training speed.

**Preprocessing Enhancements:**

- Implementing context-aware negation detection (e.g., capturing "not happy" as a single negative phrase rather than treating "not" and "happy" separately).
- Adding character-level features or byte-pair embeddings to handle creative spellings and rare slang that subword tokenization might miss.

**Architectural Extensions:**

- Exploring hybrid models that combine transformer embeddings with lightweight convolutional layers, to emphasize local n-gram patterns.
- Testing deeper classifiers (e.g., additional dense layers or bottleneck layers) on top of the [CLS] representation to see if more hierarchical modeling improves separation of borderline cases.
- Investigating knowledge distillation from even larger teacher models (e.g., RoBERTa or XLNet) into smaller student variants to push accuracy further while retaining efficiency.

**Final comparison:** Both models handle Twitter sentiment effectively, but BERT consistently outperforms DistilBERT by a narrow margin across all evaluation metrics. BERT's larger capacity allows it to capture subtler contextual cues, reflected in its slightly higher validation accuracy and AUC, whereas DistilBERT, with fewer layers, reaches almost the same level of discrimination with far less computational overhead. In practice, BERT's steeper initial drop in training loss paired with a more pronounced rise in validation loss by epoch three suggests it benefits from additional regularization to avoid overfitting. DistilBERT, on the other hand, maintains a steadier validation trajectory, indicating greater inherent robustness to noisy tweet text despite its reduced size. Ultimately, when raw accuracy is the priority, BERT's marginal gains justify its heavier footprint; conversely, when inference speed or memory constraints matter, DistilBERT delivers nearly equivalent performance with significantly lower resource requirements.

#### *4.1.1. Best Validation Trials:.* .

| Model | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| BERT | 0.8550 | 0.8615 | 0.8461 | 0.8537 |
| DistilBERT | 0.8494 | 0.8575 | 0.8380 | 0.8476 |

Table 23: Best validation performance achieved by each model

**Model Configurations for Best Trials:**

**BERT (bert-base-uncased)**:

- `Learning Rate = 3e-5, Weight Decay = 0.01, Dropout = 0.2`
- `Batch Size = 64, Hidden Dimension = 512`
- `3 epochs` with linear warmup and linear decay

**DistilBERT (distilbert-base-uncased)**:

- `Learning Rate = 2e-5, Weight Decay = 0.01, Dropout = 0.2`
- `Batch Size = 32, Hidden Dimension = 512`
- `3 epochs` with linear warmup and linear decay

**Test-Set Performance (Final Validation):**

| Model | Test Accuracy |
|---|---|
| BERT | 0.85683 |
| DistilBERT | 0.84805 |

Table 24: Final test-set accuracy for each model (using the best checkpoints)

**Summary:** While BERT edges out DistilBERT by a small margin in validation and test accuracy, the latter's near-comparable performance, combined with its reduced model size and faster inference makes it a compelling alternative when deployment efficiency is paramount. Continuous refinement, whether through more nuanced preprocessing, targeted hyperparameter sweeps, or hybrid architectures, could narrow this gap further.

### 4.2. Comparison with the First Project

In Project 1, a TF-IDF + Logistic Regression pipeline served as our sentiment baseline, requiring careful selection of vocabulary thresholds. In Project 3, we moved to fine-tuning transformer models (BERT and DistilBERT), which bypass the need for manual feature engineering. Overall, the transformer-based approach delivers noticeably stronger generalization on noisy Twitter data compared to the TF-IDF + LR setup in Project 1.

**Reasons Transformer Fine-Tuning Outperforms TF-IDF + LR:**

- ***Contextual Understanding.*** TF-IDF treats each token (or n-gram) independently, whereas transformers build embeddings that adjust dynamically based on surrounding words. This allows BERT and DistilBERT to handle negations, sarcasm, and nuanced sentiment without manually crafted bigrams.
- ***Reduced Preprocessing Overhead.*** In Project 1, extensive text cleaning (emoji mapping, punctuation tokenization, stopword tuning) was required to produce meaningful sparse vectors. In Project 3, pretrained tokenizers already normalize most noise, so we can rely on subword segmentation to capture informal language patterns with minimal custom cleaning.
- ***Robustness to Limited Data Variations.*** TF-IDF features can become sparse when a term appears infrequently, leading to brittle classifiers on short tweets. Transformers, having been pretrained on large corpora, generalize better to rare slang and misspellings, even if those variants appear only a few times in our dataset.
- ***Simplified Feature Engineering.*** TF-IDF required manually tuning n-gram ranges and document-frequency thresholds to balance noise and signal. Transformers jointly learn contextual features and classification boundaries in an end-to-end fashion, removing that trial-and-error step.
- ***Built-in Semantic Hierarchies.*** BERT's deep architecture captures multiple layers of abstraction, from surface lexicon to higher-level phrase semantics, whereas TF-IDF+LR relies solely on linear weights over sparse counts, limiting its ability to model complex sentiment constructs.

**Practical Trade-Offs Compared to Project 1:**

- ***Training Time and Resources.*** In Project 1, TF-IDF vectorization and Logistic Regression training completed in minutes on CPU, enabling rapid iteration. In Project 3, fine-tuning BERT or DistilBERT requires multiple GPU hours, significantly increasing compute costs and slowing down experiment cycles.

- ***Pipeline Complexity.*** Project 1's pipeline was straightforward: preprocess text, compute TF-IDF, and train a linear model. Project 3 introduces additional steps, setting up transformer tokenizers, crafting a custom classification head, scheduling linear warmup  decay, and monitoring early stopping, making the codebase more intricate and increasing hyperparameter management overhead.

- ***Reduced Manual Feature Engineering.*** Although Project 3's transformer pipeline is more complex computationally, it eliminates the need to manually tune n-gram ranges, document-frequency thresholds, and vocabulary size. This long-term reduction in feature engineering effort can be advantageous when text characteristics change, since one can reuse pretrained tokenizers without re-optimizing TF-IDF parameters.

- ***Greater Sensitivity to Overfitting.*** TF-IDF + LR in Project 1 typically produced smooth learning curves, as the linear model could be regularized easily via C-value. In Project 3, transformers exhibit sharper divergence between training and validation losses if learning rate, weight decay, or dropout are not carefully tuned, necessitating early stopping, gradient clipping, and more rigorous overfitting controls.

- ***Inference Speed and Deployment.*** In Project 1, the lightweight TF-IDF + LR model runs efficiently on CPU with low latency, making deployment trivial. BERT and DistilBERT in Project 3 demand more memory and compute at inference time, potentially increasing latency and infrastructure costs. While DistilBERT mitigates this somewhat by being smaller, both transformer variants still require GPU or powerful CPU clusters for real-time applications.

- ***Interpretability.*** TF-IDF + LR offers transparent feature weights, making it easy to inspect which words or n-grams drive predictions. Transformers are inherently more opaque, interpreting why a model classified a tweet positive or negative requires specialized techniques (e.g., attention visualization or gradient-based attributions), adding complexity for auditing and debugging.

**Overall Experience:**

- Using TF-IDF + LR in Project 1 felt reliable and transparent: training was fast, and inspecting feature weights made it easy to see which words drove sentiment decisions.

- Transitioning to Transformers introduced more complexity, fine-tuning schedules, gradient clipping, and careful learning-rate tuning, but paid off by dramatically reducing manual feature selection and capturing nuanced patterns automatically.

- I was surprised by how little additional preprocessing was needed for BERT and DistilBERT: subword tokenization handled most noise out of the box, whereas the TF-IDF pipeline required significant custom cleaning to achieve stable performance.

- While TF-IDF + LR remains a strong baseline for quick experiments or resource-constrained scenarios, transformers proved far more robust on noisy, informal

tweets and delivered smoother learning curves without hand-crafted feature adjustments.

- This comparison reinforces that a solid linear baseline is essential, but for tasks requiring deeper contextualization, transformer fine-tuning is the clear next step, even if it demands more computational resources.
- I was also surprised by how much more time and computational resources are needed to fine-tune BERT and DistilBERT compared to training a TF-IDF + LR model.

| **Metric** | **Project 1 (TF-IDF+LR)** | **Project 3 (BERT)** | **Project 3 (DistilBERT)** |
|---|---|---|---|
| Validation Accuracy | 0.809 | 0.855 | 0.849 |
| Test Accuracy | 0.808 | 0.853 | 0.847 |
| Precision | 0.805 | 0.861 | 0.857 |
| Recall | 0.812 | 0.846 | 0.838 |
| F1 Score | 0.810 | 0.853 | 0.847 |
| ROC AUC | 0.888 | 0.929 | 0.924 |

Table 25: Comparison between Project 1 and Project 3 (BERT & DistilBERT)

### 4.3. Comparison with the second project

In Project 2, we built a sentiment classifier by training Word2Vec embeddings on the Twitter corpus and feeding tweet-level vectors into a feed-forward neural network. Project 3 replaces that pipeline with fine-tuned transformer models (BERT and DistilBERT), leveraging pretrained contextual embeddings rather than training word vectors from scratch. Overall, the transformer-based approach shows clear improvements in generalization and robustness, but at the cost of greater computational complexity and tuning overhead.

**Reasons Transformer Fine-Tuning Outperforms Word2Vec + Neural Network:**

- ***Contextual Embeddings vs. Static Vectors.*** In Project 2, each token's representation is fixed once the Word2Vec model is trained, so subtle differences in context (e.g., "love" in a sarcastic tweet) can be lost. BERT and DistilBERT produce dynamic embeddings that adapt based on surrounding words, enabling the model to distinguish nuance and handle multi-word sentiment constructs (e.g., "not worth it") without relying on manual phrases.
- ***Reduced Hyperparameter Tuning for Embeddings.*** Project 2's embedding stage required dedicated experiments to choose vector size, window length, negative sampling, and min-count thresholds. In Project 3, pretrained transformers already capture a broad range of linguistic patterns, so fewer embedding-specific experiments are needed, this simplifies the pipeline and focuses tuning on fine-tuning hyperparameters (learning rate, weight decay, classifier head size).

- ***Better Handling of Rare and Noisy Tokens.*** Word2Vec embeddings can struggle with slang or rare misspellings unless those appear frequently enough in training. Transformers break tokens into subwords, allowing them to generalize across spelling variations and slang without manual normalization. As a result, subtle sentiment cues in low-frequency tokens are preserved more effectively.

- ***Joint Optimization vs. Separate Stages.*** In Project 2, embedding training and classification happen in two distinct stages, suboptimal embeddings may limit the neural network's capacity to learn. Transformers in Project 3 are fine-tuned end-to-end, allowing gradient signals from the sentiment classification objective to adjust the entire model together. This joint optimization yields richer representations tailored specifically to the sentiment task.

- ***Improved Generalization with Pretraining.*** The Word2Vec model in Project 2 learns solely from our tweet corpus, which can be noisy and limited in variety. Transformers start from weights pretrained on vast, diverse text, so they already internalize grammar, syntax, and common sentiment patterns. Fine-tuning on tweets then refines these pretrained patterns, leading to more robust performance on unseen data.

**Practical Trade-Offs Compared to Project 2:**

- ***Training Time and Resources.*** Project 2's Word2Vec + neural network training ran quickly on CPU or modest GPU resources. In Project 3, fine-tuning BERT or DistilBERT requires multiple epochs on a GPU, leading to longer training times. This increased resource requirement is a notable drawback if rapid iteration is needed.

- ***Pipeline Complexity.*** The Word2Vec pipeline involved a clear, two-step process: embedding training followed by classifier training. Fine-tuning transformers adds complexity: choosing appropriate learning rate schedules, implementing gradient clipping, and monitoring for overfitting. Project 3's codebase is more intricate, requiring careful management of hyperparameters to avoid instability.

- ***Reduced Manual Feature Engineering.*** Although the transformer pipeline is more complex, it eliminates the manual search for optimal embedding parameters required in Project 2. This reduction in feature engineering effort can save time in the long run, especially as modifications to data preprocessing or classifier architecture no longer require re-training embeddings from scratch.

- ***Greater Sensitivity to Overfitting.*** Word2Vec embeddings in Project 2 tended to produce smoother learning curves, since the embedding stage was not directly influenced by the classification loss. In Project 3, transformers show sharper divergence between training and validation losses if hyperparameters are not carefully tuned. This necessitates early stopping, weight decay, and learning-rate warmup, all of which add to the tuning burden.

- ***Inference Speed and Deployment.*** The lightweight neural network in Project 2 allowed fast inference on CPU. BERT and DistilBERT require more memory and compute at run-time, potentially impacting latency in production. DistilBERT

mitigates this somewhat by being smaller, but both transformer variants still demand more infrastructure than a Word2Vec+NN model.

**Overall Experience:**

- Confidence in Word2Vec + Neural Network: Project 2 demonstrated that a well-tuned Word2Vec embedding combined with a simple feed-forward network can achieve solid performance with minimal infrastructure, making it ideal for rapid prototyping or resource-constrained settings.
- Gains from Transformer Pretraining: Project 3 shows that, when compute allows, fine-tuning pretrained transformers surpasses Word2Vec by capturing deeper syntactic and semantic patterns, especially in noisy, slang-rich Twitter text.
- Balancing Complexity and Performance: While Project 3's transformer models require more time, GPU memory, and careful hyperparameter control, they substantially reduce manual feature engineering and deliver improved robustness. In contexts where accuracy and nuance matter (e.g., detecting sarcasm or subtle sentiment cues), this trade-off is worthwhile.
- Lesson Learned: Project 2 taught us the importance of careful embedding and classifier parameter tuning. Project 3 builds on that foundation by showing how pretrained language models can automate much of that work, but it also highlights the need for vigilant overfitting control and infrastructure readiness when moving to deep transformer architectures.

| Metric | Project 2 (Word2Vec+NN) | Project 3 (BERT) | Project 3 (DistilBERT) |
|---|---|---|---|
| Validation Accuracy | 0.786 | 0.855 | 0.849 |
| Test Accuracy | 0.790 | 0.853 | 0.847 |
| Precision | 0.776 | 0.861 | 0.857 |
| Recall | 0.805 | 0.846 | 0.838 |
| F1 Score | 0.790 | 0.853 | 0.847 |
| ROC AUC | 0.870 | 0.929 | 0.924 |

Table 26: Comparison between Project 2 and Project 3 (BERT & DistilBERT)

## 5. Links for the Notebooks

For easy access to the notebooks

**Kaggle Notebooks:**
View BERT Notebook
View DISTILBERT Notebook